

Queue Using SLL:

Void enqueue(int value)

{

Struct Node *newNode = (createNode(value);

if (rear == NULL)

temp & rear = front = newNode;

} else

{ ~~front~~ rear → next = newNode;

rear = newNode;

} printf("%d is enqueued", value);

}

Void dequeue()

{ struct Node *temp;

if (front == NULL)

{

printf("Queue is empty");

}

temp = front;

front = front → next;

printf("%d is dequeued", temp → data);

free(temp);

}

1) Trace the following instructions for DLL

1) insert 65 in the beginning

2) insert 43 in the end.

3) insert 76 in the beginning

4) insert 100 after 65

5) insert 200 after 100

6) Delete 43

7) Delete in the beginning

8) Delete in the end.

Ans 1) insert 65 in the beginning:



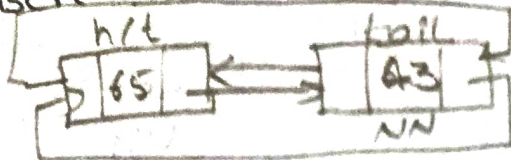
Create Node

head = tail = NULL

head ~~next~~ → prev = head

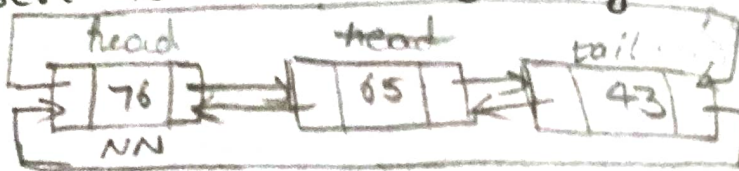
head → next = head.

2) insert 43 in the end.



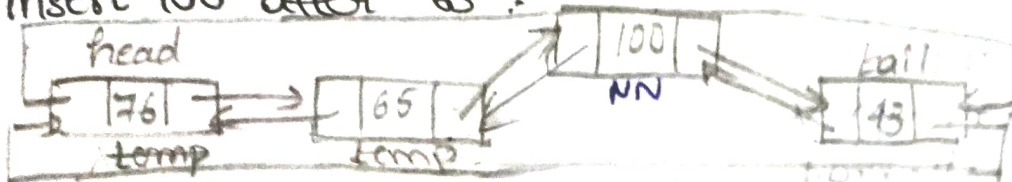
Create node NN
 $NN \rightarrow prev = head$
 $head \rightarrow prev = NN$
 $head \rightarrow next = NN$
 $NN \rightarrow next = head$
 $tail = NN$

3) insert 76 in the beginning



$NN \rightarrow next = head$
 $head \rightarrow prev = NN$
 $NN \rightarrow prev = tail$
 $tail \rightarrow next = NN$
 $head = NN$

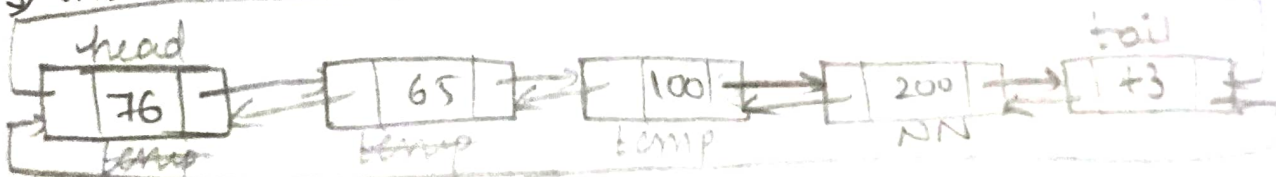
4) insert 100 after 65 :



Key = 65
 if ($temp \rightarrow data == Key$)
 $temp = temp \rightarrow next;$

$temp \rightarrow next = NN$
 $NN \rightarrow next = tail$
 $tail \rightarrow prev = NN$
 $NN \rightarrow prev = temp$

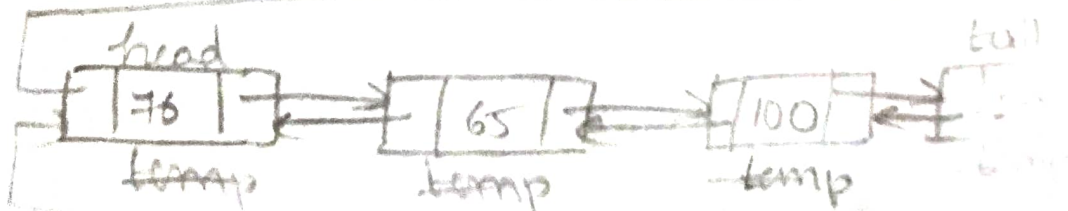
5) insert 200 after 100



Key = 100
 if ($temp \rightarrow data != Key$)
 $temp = temp \rightarrow next;$

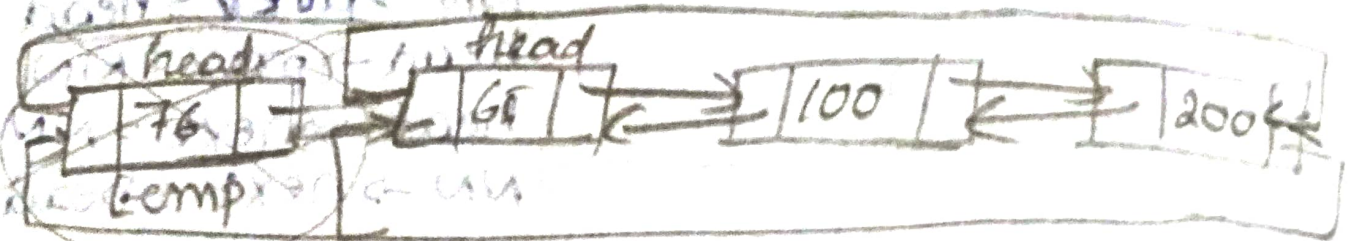
$temp \rightarrow next = NN$
 $NN \rightarrow prev = temp$
 $NN \rightarrow next = temp \rightarrow next$
 $temp \rightarrow next \rightarrow prev = NN$

6) delete 43



$tail = tail \rightarrow prev$
 $tail \rightarrow next = head$
 $head \rightarrow prev = tail$
 $free(temp)$

7) Delete in the beginning.



temp = head

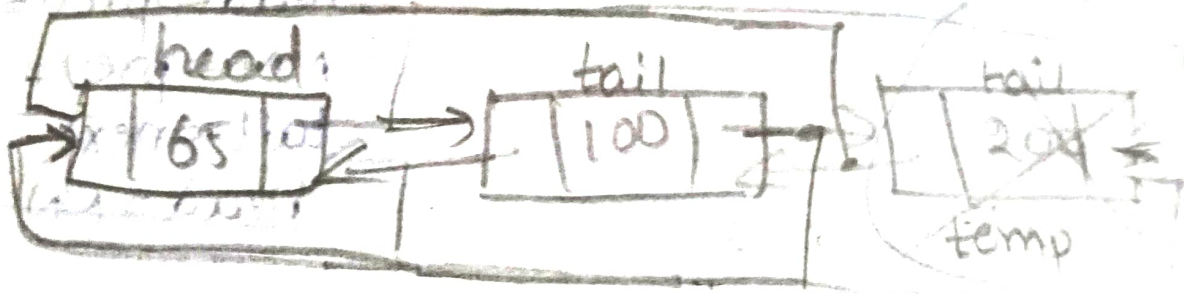
head = head → next

head → prev = tail

tail → next = head

free(temp);

8) delete in the end.



temp = tail

tail = tail → prev

tail → next = head

head → prev = tail

free(temp);